```
1  // Programmer: Mihalis Tsoukalos
2  // Date: Monday 27 May 2013
3  //
4  // File: countLines.go
5  //
6  // This Go program counts the number of lines in a
text file
7
8  package main
9
10 import (
11     "os"
12     "bufio"
13     "io"
14     "fmt"
15      "path/filepath"
16 )
17
18 func main() {
19
20         if (len(os.Args) == 1) {
21                 fmt.Printf("usage: %s <file1> [<file2>
[... <fileN]]\n",
22                                         filepath.
Base(os.Args[0]))
23                 os.Exit(1)
24         }
25
26         for _, filename := range os.Args[1:] {
27                 fmt.Printf("%s: ", filename)
28                 countLines(filename)
29         }
30 }
31
32 func countLines(filename string) {
33         var err error
34         var numberOfLines int
35         numberOfLines = 0
36
37         f, err := os.Open(filename)
38           if err != nil {
39             fmt.Printf("Error opening file %s", err)
40             os.Exit(1)
41           }
42           defer f.Close()
43           r := bufio.NewReader(f)
44           for {
45             _, err := r.ReadString('\n')
46
47                 // If you reach the end of file
48                 // read no more
49                 if err == io.EOF {
50                   break
51               } else if err != nil {
52                         fmt.Printf("error reading file
%s", err)
53               }
54                   numberOfLines++
55           }
56
57         fmt.Printf("%d lines\n", numberOfLines)
58   }
```

You should compile it using the following command:

```
$ go build countLines.go
```

Two runs of the created executable file created the following output:

```
$ ./countLines hello2.go countLines.go
hello2.go: 8 lines
countLines.go: 58 lines
$ ./countLines
usage: countLines <file1> [<file2> [... <fileN]]
```

The line-numbers are added in order to better refer to the Go code and need not be typed. I am now going to explain the most important parts of the code:

- Lines 1-6: Comments for the program.
- Line 8: The initialisation and execution of a Go program always begins with the *main* package. After some initialisation procedures, the *main()* function gets called and the program execution begins.
  - Line 10: An *import* statement with multiple values. The *import* command is used for including the functionality of already programmed packages, including packages from the standard library.
  - Line 11: The *os* package provides platform-independent operating system variables and functions. The *os.Args* variable, which is of type *[]string* (a slice of strings) holds the command line arguments. Its length can be determined using the *len()* function.
  - Line 12: The *bufio* package provides functions for buffered I/O, including functions for reading and writing UTF-8 encoded text files.
  - Line 13: The *io* package provides low level I/O functions.
  - Line 14: The *fmt* package provides functions for formatting text and for reading formatted text.
  - Line 15: The *path/filepath* package provides functions for dealing with filenames and paths. The functions are platform-independent.
  - Line 26: The := operator is used for *both* declaring and initialising a variable. This is called a *short variable declaration* in Go terminology.
  - Line 32: The *countLines()* function reads the text